

SAE3.02 : DEVELOPPEMENT DES APPLICATIONS COMMUNICANTES

Baye Niass GUEYE

SOMMAIRE

- I. Éléments implémentés et non implémentés**
- II. Structure du code, modules, protocole, API**
- III. Description de l'algorithme de chiffrement
(forces et faiblesses)**
- IV. Rapport de projet avec gestion du projet**

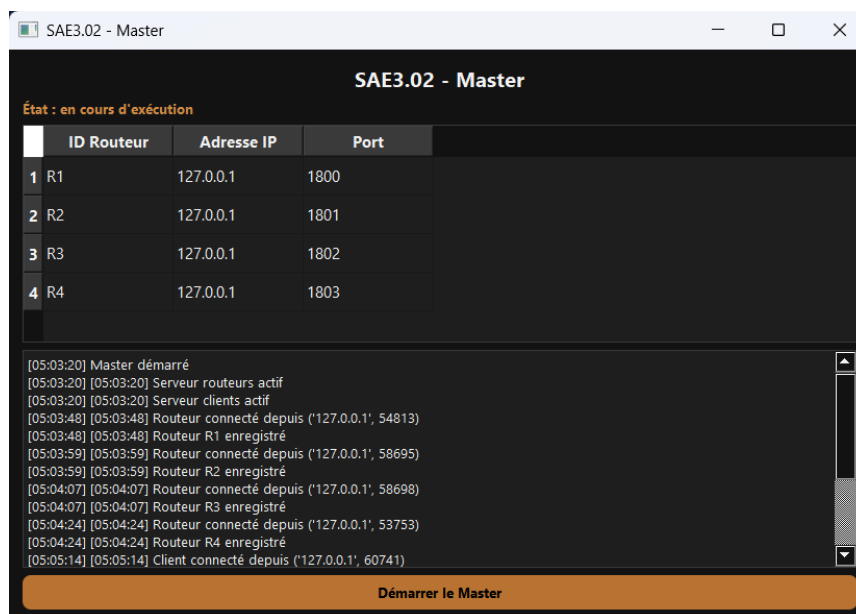
Cette SAE avait pour objectif de concevoir une application client/serveur sécurisée reposant sur un réseau de routeurs intermédiaires, avec un chiffrement de type RSA et une supervision centralisée via un Master.

Eléments implémentés et non implémentés

Eléments implémentés

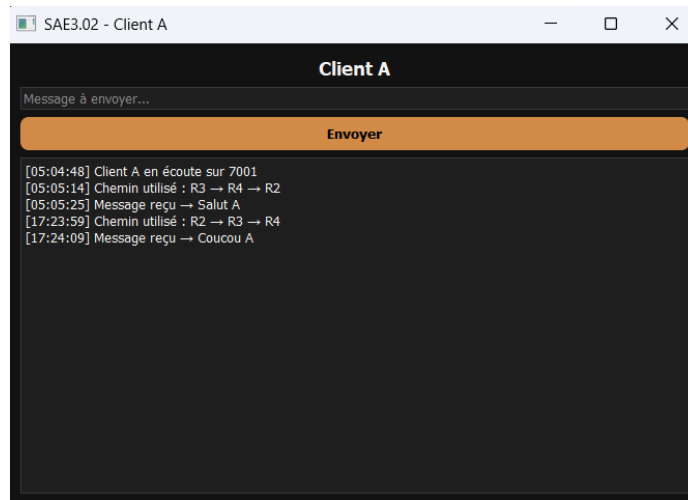
J'ai réussi à implémenter :

- Une interface graphique Master permettant la supervision des routeurs connectés, l'affichage des logs et la distribution des clés publiques.



Interface du Master

- Des routeurs intermédiaires capables de s'enregistrer automatiquement auprès du Master et de relayer les messages chiffrés.
- Un mécanisme de routage en oignon où chaque routeur ne connaît que le prochain saut.
- Un chiffrement RSA pédagogique appliqué caractère par caractère pour protéger le message final.
- Des interfaces graphiques Client A et Client B permettant l'envoi et la réception de messages de manière non bloquante.

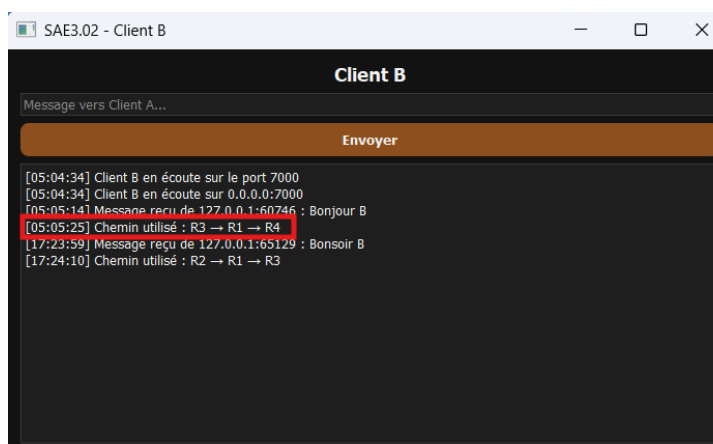


Interface du client A



Interface du client B

- L'affichage du chemin emprunté par le message côté client afin de visualiser le routage.



Éléments non implémentés

- Le chiffrement RSA implémenté reste volontairement simplifié et ne correspond pas à une implémentation cryptographiquement sécurisée utilisée en production.
- La gestion avancée des erreurs réseau (pannes de routeurs en cours de transmission) n'a pas été implémentée.
- L'authentification forte des clients et des routeurs n'a pas été mise en place.
- Le stockage persistant des messages n'a pas été réalisé.

Ces limites sont cohérentes avec le cadre pédagogique de la SAE et les contraintes de temps imposées.

Structure du code, modules, protocole, API

Organisation générale du projet

Le projet est structuré en plusieurs modules distincts afin de séparer les responsabilités et de faciliter la maintenance du code.

Tout d'abord j'ai créé trois dossiers : **Source, Documentation, et Vidéo**

Dans Source

J'ai créé trois sous-dossiers : **Client, Router, Master**

Dans chaque dossier il y a des codes propres à eux

- **master.py** dans **Master** : logique serveur central (enregistrement des routeurs, distribution des informations).
- **master_gui.py** dans **Master** : interface graphique du Master.
- **router.py** dans **Router** : routeur intermédiaire assurant le relais des messages.
- **client_oignon.py** dans **Client** : logique d'envoi des messages chiffrés via le réseau de routeurs.
- **clientA_gui.py** et **clientB_gui.py** dans **Client** : interfaces graphiques des clients A et B.

Description du protocole utilisé

La communication repose sur le protocole TCP afin d'assurer une transmission fiable des données entre les différents composants.

- Le Master écoute sur un port dédié pour les routeurs et les clients.
- Les routeurs s'enregistrent automatiquement au démarrage.
- Les clients récupèrent dynamiquement la liste des routeurs.
- Les messages sont encapsulés sous forme de couches successives (routage en oignon).

API internes (fonctions clés)

Afin de structurer le projet et de faciliter la communication entre les différents composants, plusieurs fonctions jouent le rôle d'API interne. Ces fonctions permettent d'isoler la logique réseau, le chiffrement et la communication entre les modules du projet.

demander_routeurs()

La fonction **demander_routeurs()** permet au client de se connecter au Master afin de récupérer dynamiquement la liste des routeurs disponibles.

Elle établit une connexion TCP avec le Master, reçoit les informations concernant chaque routeur (identifiant, adresse IP, port et clé publique), puis les stocke dans une structure de données exploitable par le client.

Cette approche permet au client de ne pas avoir de configuration statique et de s'adapter automatiquement aux routeurs actifs.

envoyer_message_oignon()

Cette fonction est responsable de l'envoi d'un message via le réseau de routeurs en utilisant un mécanisme de routage en oignon.

Elle sélectionne un chemin aléatoire de routeurs, chiffre le message final à l'aide de la clé publique du dernier routeur, puis encapsule successivement les informations de routage.

Le message est ensuite transmis au premier routeur du chemin, qui se charge de relayer les données jusqu'au destinataire final.

start_client_b()

Cette fonction permet de démarrer le client récepteur de manière non bloquante.

Elle lance un serveur TCP dans un thread séparé afin d'écouter les messages entrants sans bloquer l'interface graphique.

Cette fonction assure la réception des messages envoyés par les autres clients et permet leur affichage en temps réel dans l'interface graphique du client.

L'utilisation de ces fonctions comme API interne permet de séparer clairement les responsabilités du projet, d'améliorer la lisibilité du code et de faciliter l'intégration des interfaces graphiques avec la logique réseau.

Description de l'algorithme de chiffrage (forces et faiblesses)

Principe général du chiffrement utilisé

Dans ce projet, un mécanisme de chiffrement asymétrique inspiré de l'algorithme RSA a été mis en œuvre afin d'assurer la confidentialité des messages échangés entre les clients à travers les routeurs.

Chaque routeur génère une paire de clés composée d'une clé publique et d'une clé privée. La clé publique est transmise au Master lors de l'enregistrement du routeur, tandis que la clé privée reste strictement locale au routeur.

Remarque : Le chiffrement est utilisé dans le cadre d'un **roulage en oignon**, où le message est encapsulé en plusieurs couches successives de chiffrement, chacune correspondant à un routeur du chemin.

Fonctionnement du chiffrement dans le projet

Lorsqu'un client souhaite envoyer un message, il récupère la liste des routeurs disponibles ainsi que leurs clés publiques auprès du Master.

Un chemin aléatoire de routeurs est ensuite sélectionné.

Le message final est chiffré avec la clé publique du dernier routeur du chemin, puis encapsulé avec les informations de routage chiffrées pour chaque routeur intermédiaire.

A chaque étape :

- Un routeur ne peut déchiffrer **qu'une seule couche** du message,
- Il ne connaît que le **prochain saut**,
- Il ne peut ni lire le message final ni connaître le chemin complet.

Ce mécanisme permet de garantir un certain niveau d'anonymat et de confidentialité des échanges.

Forces de l'algorithme mis en place

Le principal avantage de cette approche est la séparation des connaissances entre les différents routeurs. Aucun routeur ne dispose d'une vision complète du message ou de son origine et de sa destination finale.

Les principales forces sont :

- L'utilisation d'un chiffrement asymétrique évitant le partage de clés secrètes,
- La génération dynamique des clés RSA par chaque routeur,
- Le roulage en oignon qui limite l'exposition des informations sensibles,
- La compatibilité avec une architecture distribuée et modulaire.

Cette solution est adaptée à un contexte pédagogique permettant de comprendre les concepts de sécurité réseau et de chiffrement.

Faiblesses et limites de la solution

L'algorithme de chiffrement utilisé reste volontairement simplifié et n'a pas vocation à être utilisé en production.

Les principales limites identifiées sont :

- Les tailles de clés RSA sont réduites, ce qui rend le chiffrement vulnérable à des attaques par force brute,
- Aucune signature numérique n'est utilisée pour garantir l'authenticité des messages,
- L'absence de chiffrement symétrique pour les échanges peut entraîner une surcharge de calcul,

- La gestion des erreurs et des attaques actives (rejeu, interception) n'est pas prise en compte.

Ces choix ont été faits dans un objectif pédagogique afin de privilégier la compréhension du fonctionnement global plutôt que la robustesse cryptographique.

Le mécanisme de chiffrement implémenté dans ce projet permet de démontrer les principes fondamentaux du chiffrement asymétrique et du routage en oignon.

Bien que simplifié, il répond aux objectifs du cahier des charges en assurant la confidentialité des messages et en illustrant les enjeux de sécurité dans une application réseau distribuée.

Rapport de projet avec gestion du projet

Le projet SAE 3.02 a fait l'objet d'une **gestion de projet structurée dès le début**, afin d'assurer un développement progressif, cohérent et conforme au cahier des charges.

Avant de commencer le développement, un **document de gestion de projet** a été rédigé. Celui-ci présente la vision globale du projet, les objectifs techniques, les fonctionnalités attendues ainsi que l'organisation du travail. L'objectif principal était de concevoir une application de communication sécurisée et anonyme inspirée du **routage en oignon**, reposant sur une architecture client–routeurs–client pilotée par un master